

The **algebra axioms**, and what defines them in the **Frobenius calculus**

Freyd, *Categories, Algebras* §3.11-§2.10 against functorialSemanticsForRelationalTheories.pdf (v14.1)

Lean: Freyd/S2_10.lean(algebra,diag/CB.lean(CartBicat,diag/CB.Algebra,Lean(algebraOfCartBicat))

Where the pictures come from: Every picture of a theorem is emitted by ./scripts/diag-export from that theorem's Lean type. Nothing after a semicolon is returned here. The two figures marked *untranscribed* are the exception: they copy the paper's definition pictures, and definition has no statement to export.

Equation numbering. The numbers below are the 2017 preprint's. The CartBicat fields no longer encode them – a field is named for what it says, $\langle \text{R} \rangle \leq \text{I}$ rather than 1InEq_37 and each desctrictor carries the preprint number. The repo's copy of the paper is now the published version – Pops, J. Math. 2 (6): 30 Oct 2025, doi:10.67803/polymath.16 – which numbers. Translation, published ← preprint: (34)–(37), (35)–(38), (39)–(39), (37)–(40), (38)–(41), (39)–(42), (40)–(43), (35)–(35), and Example 2.3(c)'s (13)(14)–(11)(12); the preprint's (56) is unnumbered there. Pages: Def. 4.1 at 20–21 (was 17), the ∇ notation at 21 (was 19), Lemma 4.2 at 21–22 (was 19). Renumbering the Lean fields is a separate decision and has not been made.

Freyd's cartesian bityakes reciprocation * and intersection $\cap$ as **primitive operations** and lists eight **equational axioms** on them. A cartesian bityake of relations takes neither: every object carries one special Frobenius structure $\Delta, \text{I}, \nabla, ?$, the hom-ord ∇ is primitive, and both * and $\cap$ are **defined** from the four dots. Every one of Freyd's eight axioms is then a theorem. The table below is that dictionary, one row per axiom.

Reading the pictures: left to right is composition – the book's diagram order, R $\circ$ S means first R then S – vertical stacking is $\otimes$, a bare wire is an identity, a white box is relation, and a box that is mirrored and tinted is its converse – the mirroring is the reciprocal's own notation, the tint is this note's, because at a glance a chamfer is easy to miss. The label inside is not mirrored: the paper can reverse a single letter and still be read, a whole term like R $\cup$ S cannot. All four dots are solid, as Def. 4.1 draws them. The exporter draws a meet or a converse as a **labelled box**, so a composite under a * appears as a single box reading R ; S ; a $\otimes$ draws as two stacked nodes, and any subterm with no shape in this calculus – an associator, say – degrades to an opaque box. The term columns write the converse *, Freyd's symbol, where the paper writes RT: these terms are read against the box throughout, and one symbol per idea beats asking the reader to match two. The Lean sources write R* as well. The paper reserves (-) for the colour swap of its §7, but §7 is a worked example in other modes – see the caution at the end of this note – and nothing here formalises it, so in a repo that is R*(Set) throughout there is no clash to protect against.

In a chain table the symbol at the head of each row is the relation that **step** was proved at: for a reshaping, $\leq$ for a genuine inequality. Counting them the point of finding a proof at all – a chain of one $\leq$ between equalities says the theorem is one law and a lot of bookkeeping.

1. Freyd's primitives are the Frobenius calculus's definitions

Freyd's primitive, and what defines it here	picture
reciprocation $R^* \dashv \vdash R$ – §2.11, primitive. Here conv $R := \text{bend} \ (R \otimes \text{I}) ; \text{cap}$ (diag/CB.lean:223), not a generator. Input arrives bottom left and runs under R; the cup ? ; Δ at the top left creates a pair, one strand feeding R and the other leaving as the output at the top right; the cap ∇ ; I at the bottom right annihilates R's output against the input. It is the cup and the cap that reverse the arrow – a wire neatly bumped up over the box and back down is an isotopy of R itself.	
intersection $R \cap S$ – §2.11, primitive. Here the convolution meet $R \cap S := \Delta ; (R \otimes S) ; \nabla$ (diag/CB.Derived.lean:24). Copy the input, run both, merge; the merge forces the two results to agree.	
containment $R \subseteq S$ – §2.11, defined as $R \cap S = R$. Here the poset-enrichment $\leq$ is primitive and $\cap$ is derived, so the two directions are swapped. meet_eq_left_of_le (diag/CB.Derived.lean:196) is the bridge. maximal morphism $\dashv \vdash$ – §2.15, and Freyd needs a unitary allegory for it. Here top a $\text{b} := \text{!} ; ?$ (diag/CB.Derived.lean:28) is free in Def. 4.1. $\text{Freyd.Dia}g.\leq_{\text{top}}$.	

2. The eight axioms, and what proves each

Freyd's axiom, and what supplies it	picture, from the Lean statement
$(R^*)^* = R$ – §2.11 recip_recip; here the snake , from (41). Freyd.Dia.g.CartBicat.conv_conv. Bending down and back up straightens out, so bending twice is the identity.	
$(R \cap S)^* = R^* \cap S^*$ – §2.11 recip_comp; here mirroring the picture. Freyd.Dia.g.CartBicat.conv_conv. Reflecting a chain left to right reverses it; each box comes back flipped.	
$(R \cap S)^* = R^* \cap S^*$ – §2.11 recip_inter; the same mirroring. Freyd.Dia.g.conv_inter. Δ and ∇ are each other's mirror image, so reflecting the convolution leaves its shape alone and only turns the boxes round.	
$R \cap R = R$ – §2.11 inter_idem; here (42) and the special law. Freyd.Dia.g.meet_idem. Copy, run R twice, merge – (42) pushes R back through the copy and Δ ; $\nabla := \text{I}$ closes the bubble.	
$R \cap S = S \cap R$ – §2.11 inter_comm; here commutativity (9) and commutativity (6). Freyd.Dia.g.meet_comm. The two strands of Δ are interchangeable, and so are those of ∇ .	
$R \cap (S \cap T) = (R \cap S) \cap T$ – §2.11 inter_assoc; here coassociativity (8) and associativity (5). Freyd.Dia.g.meet_assoc. Either bracketing builds the same three-way copy tree, and dually the same merge tree.	
semi-distributivity $R (S \cap T) \subseteq R \cap S \cap R T$ – §2.11 semidistrib. §2.136; here (42), the lax comonoid inequation. Freyd.Dia.g.semidistrib_of_lax. Running R and then copying is below copying and then running R on each strand.	
modular identity $R \cap S \cap T \subseteq (R \cap S T) S$ – §2.112 modular; the axiom Freyd adjoins: "we adjoin one more containment, the law of modularity". Here (41) and (42), derived. Freyd.Dia.g.modular_of_frobenius. Copy the shared prefix Δ ; $(R \otimes T)$ off both sides and what is left is $\leq \nabla \text{slide_conv}$ (diag/CB.lean:566) – a box slides through a merge at the cost of its converse on the other strand.	

3. The other direction: what each law of Def. 4.1 says in Freyd's language

picture, from the Lean statement	Def. 4.1	Freyd's
	$(37) \nabla ; \Delta \leq \text{I}$ half of $\Delta \dashv \nabla$	$\Delta^* \Delta \dashv \text{I} = \Delta$ is simple §2.13. Not used anywhere in the bridge; see below.
	$(38) \text{I} \leq \Delta ; \nabla$ the other half of $\Delta \dashv \nabla$	$\text{I} \subseteq \Delta \Delta^* = \Delta$ is entire §2.13; with (39), $\text{I} \subseteq \text{cap}$. This is what makes $\text{f} = \text{I} ; ?$ contain every arrow.
	$(39) ? ; \text{I} \leq \text{I}$ half of $\text{I} \dashv ?$	$\text{I}^* \text{I} \subseteq \text{I} = \text{I}$ is simple §2.13. Also unused in the bridge.
	$(40) \text{I} \leq \text{I} ; ?$ the other half of $\text{I} \dashv ?$	$\text{I} \subseteq \text{I}^* \text{I}^* = \text{I}$ is entire §2.13; with (39), $\text{I} \subseteq \text{cap}$. This is what makes $\text{f} = \text{I} ; ?$ contain every arrow.
	(41) the Frobenius law as an axiom: $\text{I} \otimes \Delta$ and $\nabla \otimes \text{I}$ draw as stacked nodes; only the associator α^{-1} stays an opaque box, having no shape in this calculus.	the modular identity §2.112 – and reciprocation §2.11 itself, since the cup and cap only satisfy the snake because of (41).
	the special law not an axiom: derived from (38) and (40). Freyd.Dia.g.CartBicat.special	$\Delta^* = \text{I} = \Delta$ is injective , so by §2.141 at $\text{f} = \text{g} = \Delta$ it is a monic map. In Rel(Set): copying is total and loses nothing.
	(42) every arrow is a lax comonoid homomorphism	§2.136's $R (S \cap T) \subseteq R \cap S \cap R T$, read at $S := \text{cap}^*$, $T := \text{cap}^*$ – Freyd's semi-distributivity, product-free.
	(43) every arrow is lax for the counit	§2.152's $\text{step}^* \text{p}_\alpha$ is maximal in (α, λ) , hence $R \text{p}_\beta \subseteq \text{p}_\alpha \text{p}_\gamma^*$ – that is, $R \subseteq T$.

4. Unitary and pre-tabular – the two conditions the converse needs

Freyd's §2.11 axioms are all a cartesian bityake has to give back. Going the other way needs two further conditions, and they are exactly the two the repo has not formalised. Both definitions below are quoted from Freyd/S2_10.lean, which follows the book.

condition	Freyd	what the Frobenius side already has
unitary §2.15 $\text{f} : \text{A} \rightarrow \text{B}$ $\text{f}^* : \text{B} \rightarrow \text{A}$ $\text{f} \text{f}^* = \text{I}$ $\text{f}^* \text{f} = \text{I}$	T is a partial unit when I is the largest endomorphism on it, $\text{PartialUnit } \text{I} := \nabla \text{R} : \text{T} \rightarrow \text{T}$, $R \subseteq \text{I} \text{ T}$ (:539); it is a unit when moreover every object carries an entire arrow to it, $\text{IsUnit } \text{T} := \text{PartialUnit } \text{T} \text{ A}$ $\forall \text{ a}$, $\exists \text{ R} : \text{a} \rightarrow \text{T}$, $\text{Entire } \text{R}$ (:545). A unitary allegory is an allegory with one (:548). The point of a unit is that it makes $\text{T}_{\{a,b\}}$ exist: $\text{T} \text{p}_\alpha \subseteq \text{p}_\beta$ through the unit, which is where §2.152's $\text{R} \text{p}_\beta \subseteq \text{p}_\alpha$ – the repo's (43) – comes from.	already there, for free. The monoidal unit I is the unit object. $\text{I} \otimes \text{a} := \text{a}$ – I is the entire arrow every object carries to it – (40) says so – and $\text{T} = \text{I} ; ?$ is maximal by Freyd.Dia.g.<_top>. So every cartesian bityake of relations is unitary, with nothing assumed. Not recorded in Lean: allegoryOfCartBicat builds a plain Allegory, and UnitaryAllegory instance is derived from CartBicat.
pre-tabular §2.165 $\text{f} : \text{A} \rightarrow \text{B}$ $\text{f}^* : \text{B} \rightarrow \text{A}$ $\text{f} \text{f}^* = \text{I}$ $\text{f}^* \text{f} = \text{I}$	f , g tabulate R when both are maps, $\text{R} = \text{f}^* \text{g}$, and $\text{f}^* \text{f} = \text{I}$ (Tabulates, :502); R is tabular when some pair tabulates it (:506). An allegory is PreTabular if $\text{f}^* \text{f} = \text{I}$ for all f (PreTabular, :506). Freyd.Dia.g.PreTabular. Both are built, so the round-trip. The two notations say which layer a picture is in. A union is a tape : the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a biadjoint on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a second composition with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.	this is what $\otimes$ is. Given unitarity, the condition reduces to tabulating each $\text{T}_{\{a,b\}}$ and the tabulation of $\text{T}_{\{a,b\}}$ is precisely an object $\otimes \text{b}$ with its two projections – the monoidal product Def. 4.1 assumes. That is the whole content of the swap: Freyd derives the product from tabulations, Def. 4.1 posits it. Not built: Rebuilding $\otimes$ from tabulations and products (Map(A)) is CartBicatOfUnitaryPretabular; diag/PLAN.md holds a bar for want of a consumer.

Together they are the whole gap: cartesian bityake of relations $\equiv$ unitary pre-tabular allegory (DiagrammaticAlgebraOfFirstOrderLogic §10, after Carboni-Wallers). One half of that equivalence is allegoryOfCartBicat and is a theorem in this repo; the other half is the two rows above.

5. Union and residual – what phases 8 and 9 add

u and / are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition** with linear products (diag/F0.lean, after DiagramaticAlgebraOfFirstOrderLogic.pdf). Both are built, so the round-trip.

The two notations say which layer a picture is in. A union is a **tape**: the bounded wrapper is the second monoidal product, and its $\otimes$ and $\cap$ are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a **biadjoint** on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a **second composition**